

Un solveur SAT pour Meowdoku

MLSF – Projet, partie I

Salomé Fonvielle Hugo Vigna

Octobre 2026

Plan

1. Modéliser Meowdoku en CNF
2. DIMACS et DPLL
3. Résoudre une grille
4. Heuristiques
5. Comparaison
6. Solveur optimisé
7. Usage de l'IA

Le jeu et ce qui est demandé

Règles

Grille $N \times N$, N régions, N chats (on se restreint à $N = M = K$) :

- au plus un chat par ligne et par colonne ;
- exactement un chat par région ;
- aucun chat collé à un autre (8-voisinage).

Demandé

- 1 Grille $\rightarrow$ DIMACS (variables $x_{i,j}$) dont toute valuation témoin est une solution.
- 2 Générateur de grilles à solution garantie.

Fichiers : `meowdoku.py`, `generator.py`

A	B	C	B	B	D
A	B	B	B	B	D
A	A	B	D	D	D
A	A	A	E	D	D
A	A	A	A	D	D
F	A	D	D	D	D

grille du sujet

A	B		B	B	D
A	B		B		D
A			D	D	D
A	A	A		D	D
A	A	A	A	D	
F	A	D	D	D	D

une solution

Une variable par case

$x_{i,j}$ vraie ssi un chat est en (i,j) . DIMACS numérote à partir de 1 : on cherche une bijection

$$\varphi : \llbracket 1, N \rrbracket^2 \rightarrow \llbracket 1, N^2 \rrbracket$$

$$\varphi(i, j) = (i - 1)N + j$$

$$\varphi^{-1}(k) = (\lfloor \frac{k-1}{N} \rfloor + 1, (k-1) \bmod N + 1)$$

Dans le code (indices à partir de 0)

```
def var(i, j, n):          # = phi(i+1, j+1)
    return i * n + j + 1
```

	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$
$i=1$	1	2	3	4	5	6
$i=2$	7	8	9	10	11	12
$i=3$	13	14	15	16	17	18
$i=4$	19	20	21	22	23	24
$i=5$	25	26	27	28	29	30
$i=6$	31	32	33	34	35	36

numérotation pour $N = 6$

- ❶ **Conflit** : $\neg x_a \vee \neg x_b$ pour toute paire $\{a, b\}$ sur la même ligne, la même colonne, dans la même région, ou adjacente (8-voisinage).
- ❷ **Région** : $\bigvee_{a \in R_k} x_a$ pour toute région R_k .

Règle du sujet	Clauses
au plus un chat par ligne	conflit « même ligne »
au plus un chat par colonne	conflit « même colonne »
exactement un chat par région	région + conflit « même région »
aucun chat adjacent	conflit « adjacentes »

Donc **modèles de la CNF = configurations valides**.

« Au plus un » par paires : $\frac{n(n-1)}{2}$ clauses, sans variable auxiliaire ; poser a rend chaque $\neg a \vee \neg b$ unitaire (propagation). Avec $N = M = K$, « exactement un par ligne/colonne » en découle.

Exemple : la grille du sujet

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30
31	32	33	34	35	36

numéro de variable de chaque case

p cnf 36 309

36 variables, 309 clauses

-1 -2 0

pas de chat en 1 *ou* pas en 2 (ligne)

-1 -7 0

pas de chat en 1 *ou* pas en 7 (colonne)

...

303 clauses de conflit

1 7 13 ... 32 0

au moins un chat dans A

3 0

C n'a qu'une case : chat en 3

22 0

E : chat en 22

31 0

F : chat en 31

...

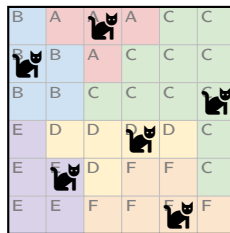
6 clauses de région

Les cases **encadrées** sont des régions d'une seule case : leur clause est unitaire, DPLL y place un chat sans décision.

Générateur : solution plantée

- 1 **Placer les chats** : une colonne par ligne, toutes distinctes, avec un écart ≥ 2 entre lignes consécutives (backtracking aléatoire). Impossible pour $N = 2, 3$.
- 2 **Faire grossir les régions** : chaque chat est la graine d'une région. Tant qu'il reste une case libre, on en tire une au hasard parmi celles qui touchent une région (4-voisinage) et on l'y rattache.

Les régions obtenues sont connexes et contiennent chacune exactement un chat. La configuration plantée est donc valide.



$N = 6$, graine 1

- `is_valid` vérifie les règles **directement**, sans passer par la CNF.
- Grilles générées pour $N = 4..8$, 20 graines chacune : la solution plantée est valide, elle satisfait la CNF, et les régions sont connexes.
- **Équivalence exhaustive** pour $N = 4, 5, 6$: pour chacune des $N!$ permutations, « satisfait la CNF » $\Leftrightarrow$ « `is_valid` ».
- Aller-retour fichier : `write_grid` puis `parse_grid` redonne la grille.

- $N = M = K$ uniquement.
- La solution n'est **pas unique** : 20 solutions pour la grille $N = 6$, graine 1. Le sujet n'exige qu'une solution au moins.
- Encodage par paires : $O(|R|^2)$ clauses par région ; une grande région coûte cher.
- Piste pour la Q6 : ajouter les clauses redondantes « au moins un par ligne/colonne » pour obtenir plus de propagation unitaire.

À compléter

À compléter

À compléter

À compléter

À compléter

À compléter